Semester	B.E. Semester VII – Computer Engineering
Subject	R Programming Lab

Student Name	Prasana Shangloo		
Roll Number	23102A0076		
Grade		Teacher's Signature	

Experiment No:	1	
Experiment Title	Data Cleaning and Missing Value Analysis Using R Programming	
Resources / Apparatus Required	Hardware: PC	Software: R, RStudio, Beijing Multi-Site Air Quality Dataset (CSV).
Objectives	To import and inspect a real-world air quality dataset in R. • To understand the concepts of NA, NULL, and NaN. • To identify and summarize missing values in the dataset. • To replace missing numerical values using the median. • To replace missing categorical values using the mode. • To implement error handling using tryCatch(). • To compare missing values before and after cleaning. • To visualize the cleaning process using a bar chart. • To export the cleaned dataset as a CSV file.	
Theory	Before any data analysis is carried out, the dataset should be cleaned to ensure that the information is accurate and reliable. Missing or incorrect values can affect the quality of the results, making data cleaning an important part of the preprocessing stage. In R, different types of missing or invalid values are represented using NA, NaN, and NULL, each serving a different purpose. In this experiment, missing values in numerical attributes are replaced using the median, while missing values in the categorical attribute are filled using the mode. User-defined functions, loops, and tryCatch() are also used to simplify the cleaning process and handle possible errors. Finally, the original and cleaned datasets are compared, and the results are presented using a grouped bar chart to show the improvement after data cleaning.	
Screenshot	**Task 1**	

	Imported the dataset using read.csv() with tryCatch(), displayed the first six records, checked the structure, dimensions, and missing values.

```
################################################################
############
# TASK 1: IMPORT DATASET, INSPECT STRUCTURE, CHECK MISSING VALUES
################################################################
############

file_path <- "data/PRSA_Data_Aotizhongxin_20130301-20170228.csv"

air_quality_data <- tryCatch({
  read.csv(file_path, stringsAsFactors = FALSE)
}, error = function(e) {
  message("Error while reading the file: ", conditionMessage(e))
  return(NULL)
}, warning = function(w) {
  message("Warning while reading the file: ", conditionMessage(w))
  return(NULL)
})

cat("\n----- Task 1: Dataset Import and Inspection -----\n")

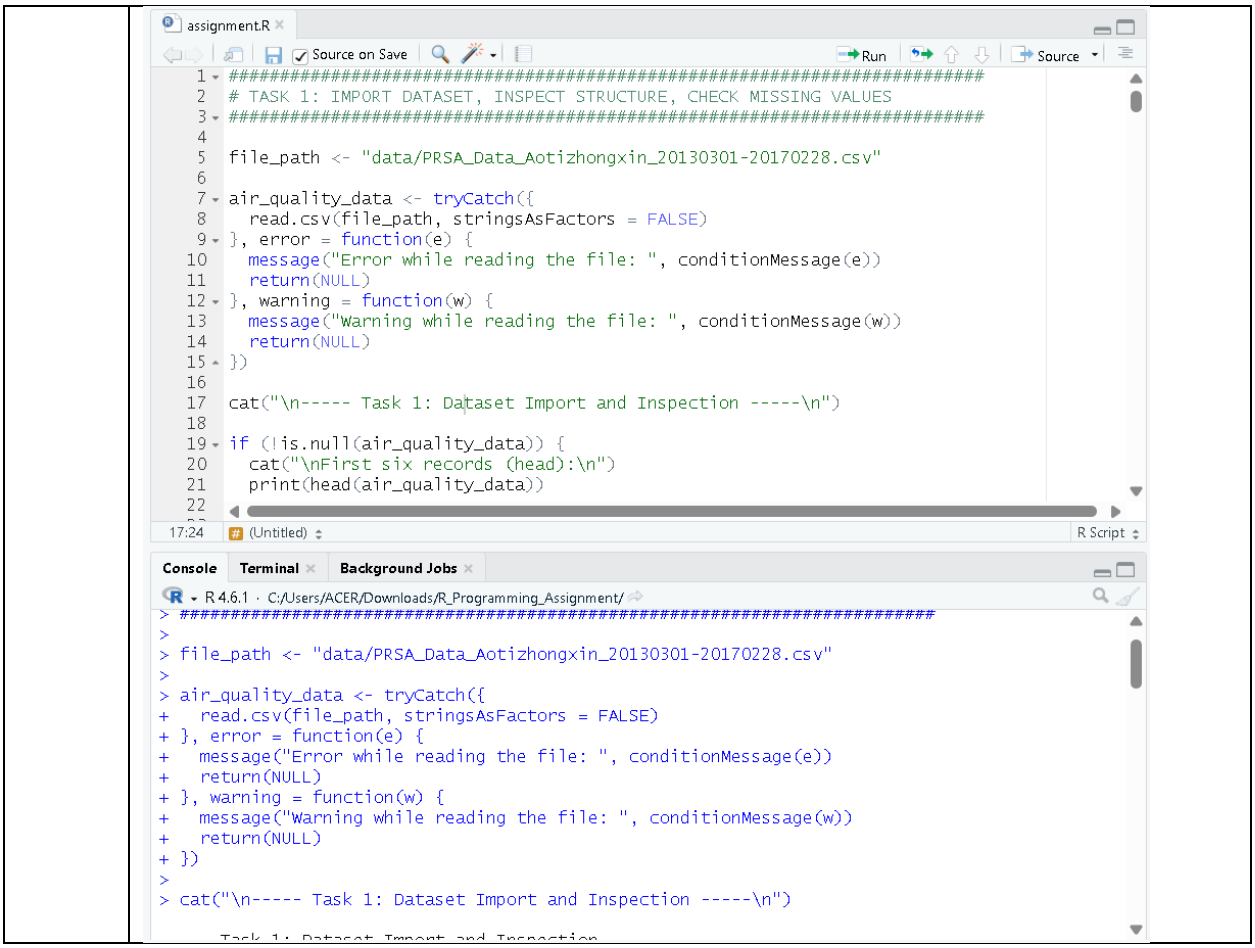
if (!is.null(air_quality_data)) {
  cat("\nFirst six records (head):\n")
  print(head(air_quality_data))

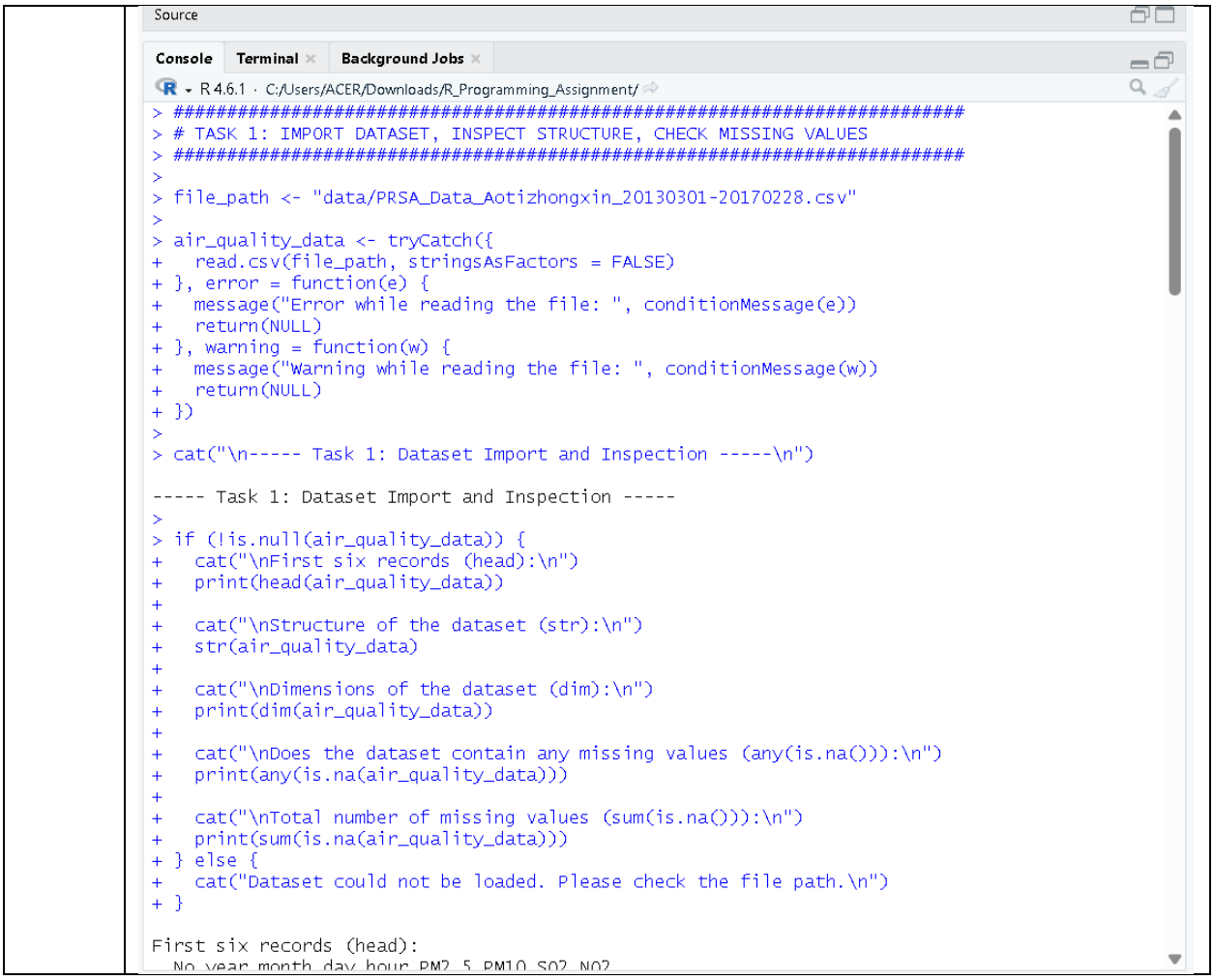
  cat("\nStructure of the dataset (str):\n")
  str(air_quality_data)

  cat("\nDimensions of the dataset (dim):\n")
  print(dim(air_quality_data))

  cat("\nDoes the dataset contain any missing values (any(is.na())):\n")
  print(any(is.na(air_quality_data)))

  cat("\nTotal number of missing values (sum(is.na())):\n")
  print(sum(is.na(air_quality_data)))
} else {
  cat("Dataset could not be loaded. Please check the file path.\n")
}
```


assignment.R
Source on Save
Run
Source
1 ##
2 # TASK 1: IMPORT DATASET, INSPECT STRUCTURE, CHECK MISSING VALUES
3 ##
4
5 file_path <- "data/PRSA_Data_Aotizhongxin_20130301-20170228.csv"
6
7 air_quality_data <- tryCatch({
8 read.csv(file_path, stringsAsFactors = FALSE)
9 }, error = function(e) {
10 message("Error while reading the file: ", conditionMessage(e))
11 return(NULL)
12 }, warning = function(w) {
13 message("Warning while reading the file: ", conditionMessage(w))
14 return(NULL)
15 })
16
17 cat("\n----- Task 1: Dataset Import and Inspection -----\n")
18
19 if (!is.null(air_quality_data)) {
20 cat("\nFirst six records (head):\n")
21 print(head(air_quality_data))
22
17:24
(Untitled)
R Script
Console
Terminal
Background Jobs
R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/
> ##
>
> file_path <- "data/PRSA_Data_Aotizhongxin_20130301-20170228.csv"
>
> air_quality_data <- tryCatch({
+ read.csv(file_path, stringsAsFactors = FALSE)
+ }, error = function(e) {
+ message("Error while reading the file: ", conditionMessage(e))
+ return(NULL)
+ }, warning = function(w) {
+ message("Warning while reading the file: ", conditionMessage(w))
+ return(NULL)
+ })
>
> cat("\n----- Task 1: Dataset Import and Inspection -----\n")


Source
Console
Terminal
Background Jobs
R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/
> ##
> # TASK 1: IMPORT DATASET, INSPECT STRUCTURE, CHECK MISSING VALUES
> ##
>
> file_path <- "data/PRSA_Data_Aotizhongxin_20130301-20170228.csv"
>
> air_quality_data <- tryCatch({
+   read.csv(file_path, stringsAsFactors = FALSE)
+ }, error = function(e) {
+   message("Error while reading the file: ", conditionMessage(e))
+   return(NULL)
+ }, warning = function(w) {
+   message("Warning while reading the file: ", conditionMessage(w))
+   return(NULL)
+ })
>
> cat("\n----- Task 1: Dataset Import and Inspection -----\n")

----- Task 1: Dataset Import and Inspection -----
>
> if (!is.null(air_quality_data)) {
+   cat("\nFirst six records (head):\n")
+   print(head(air_quality_data))
+
+   cat("\nStructure of the dataset (str):\n")
+   str(air_quality_data)
+
+   cat("\nDimensions of the dataset (dim):\n")
+   print(dim(air_quality_data))
+
+   cat("\nDoes the dataset contain any missing values (any(is.na())):\n")
+   print(any(is.na(air_quality_data)))
+
+   cat("\nTotal number of missing values (sum(is.na())):\n")
+   print(sum(is.na(air_quality_data)))
+ } else {
+   cat("Dataset could not be loaded. Please check the file path.\n")
+ }

First six records (head):
  No year month day hour PM2.5 PM10 SO2 NO2

Console | Terminal | Background Jobs

R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/

```
First six records (head):
  No year month day hour PM2.5 PM10 SO2 NO2
1  1 2013     3   1    0     4    4   4   7
2  2 2013     3   1    1     8    8   4   7
3  3 2013     3   1    2     7    7   5  10
4  4 2013     3   1    3     6    6  11  11
5  5 2013     3   1    4     3    3  12  12
6  6 2013     3   1    5     5    5  18  18
   CO O3 TEMP   PRES  DEWP RAIN  wd WSPM
1 300 77 -0.7 1023.0 -18.8    0 NNW  4.4
2 300 77 -1.1 1023.2 -18.2    0   N  4.7
3 300 73 -1.1 1023.5 -18.2    0 NNW  5.6
4 300 72 -1.4 1024.5 -19.4    0  NW  3.1
5 300 72 -2.0 1025.2 -19.5    0   N  2.0
6 400 66 -2.2 1025.6 -19.6    0   N  3.7
       station
1 Aotizhongxin
2 Aotizhongxin
3 Aotizhongxin
4 Aotizhongxin
5 Aotizhongxin
6 Aotizhongxin

Structure of the dataset (str):
'data.frame':   35064 obs. of  18 variables:
 $ No     : int  1 2 3 4 5 6 7 8 9 10 ...
 $ year   : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
 $ month  : int  3 3 3 3 3 3 3 3 3 3 ...
 $ day    : int  1 1 1 1 1 1 1 1 1 1 ...
 $ hour   : int  0 1 2 3 4 5 6 7 8 9 ...
 $ PM2.5  : num  4 8 7 6 3 5 3 3 3 3 ...
 $ PM10   : num  4 8 7 6 3 5 3 6 6 8 ...
 $ SO2    : num  4 4 5 11 12 18 18 19 16 12 ...
 $ NO2    : num  7 7 10 11 12 18 32 41 43 28 ...
 $ CO     : int  300 300 300 300 300 400 500 500 500 400 ...
 $ O3     : num  77 77 73 72 72 66 50 43 45 59 ...
 $ TEMP   : num  -0.7 -1.1 -1.1 -1.4 -2 -2.2 -2.6 -1.6 0.1 1.2 ...
 $ PRES   : num  1023 1023 1024 1024 1025 ...
 $ DEWP   : num  -18.8 -18.2 -18.2 -19.4 -19.5 -19.6 -19.1 -19.1 -19.2 -19.3 ...
 $ RAIN   : num  0 0 0 0 0 0 0 0 0 0 ...
 $ PRES   : num  1023 1023 1024 1024 1025 ...
 $ DEWP   : num  -18.8 -18.2 -18.2 -19.4 -19.5 -19.6 -19.1 -19.1 -19.2 -19.3 ...
 $ RAIN   : num  0 0 0 0 0 0 0 0 0 0 ...
 $ wd     : chr  "NNW" "N" "NNW" "NW" ...
 $ WSPM   : num  4.4 4.7 5.6 3.1 2 3.7 2.5 3.8 4.1 2.6 ...
 $ station: chr  "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" "Aotizhongxin" ...

Dimensions of the dataset (dim):
[1] 35064    18

Does the dataset contain any missing values (any(is.na())):
[1] TRUE

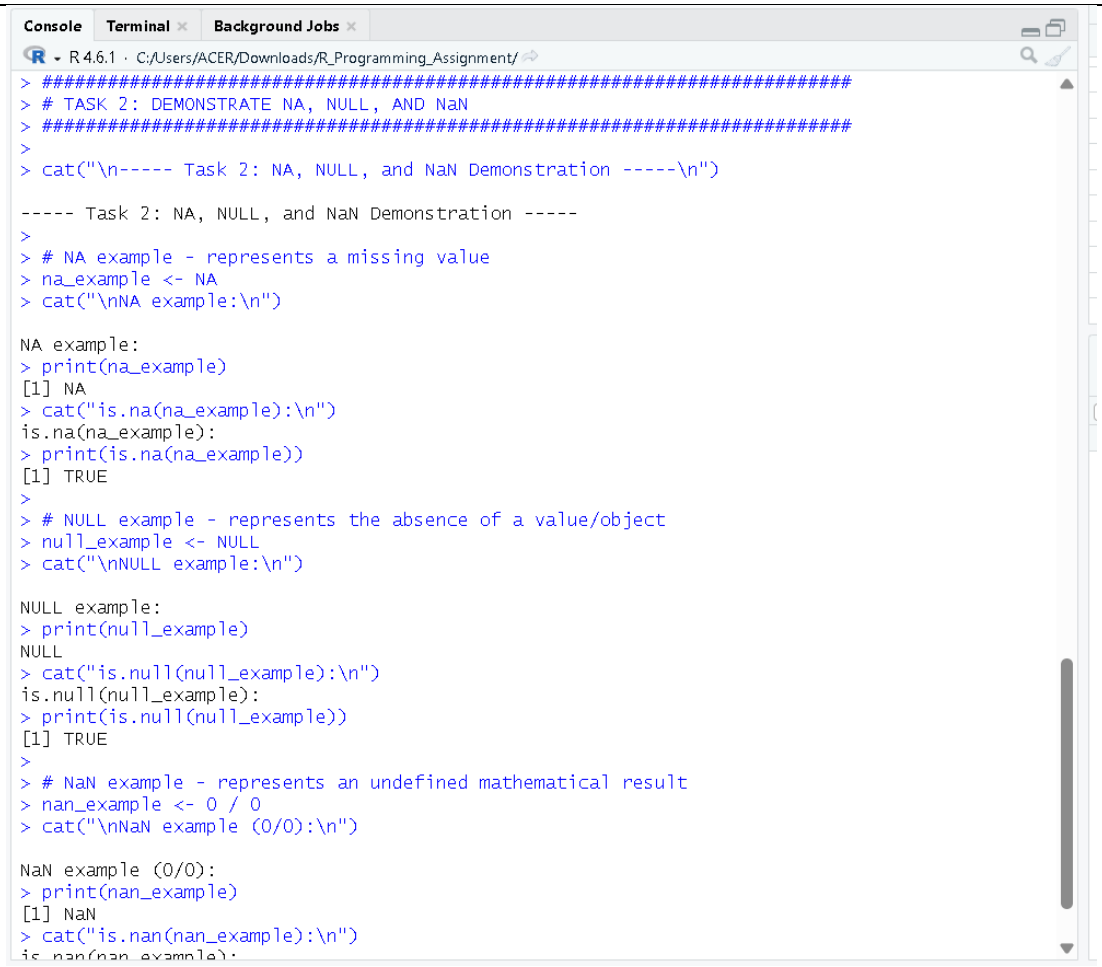
Total number of missing values (sum(is.na())):
[1] 7271
```

Task 2

Created examples of NA, NULL, and NaN and demonstrated their identification using is.na(), is.null(), and is.nan().

##
TASK 2: DEMONSTRATE NA, NULL, AND NaN
##
cat("\n----- Task 2: NA, NULL, and NaN Demonstration -----\n")
NA example - represents a missing value
na_example <- NA cat("\nNA example:\n") print(na_example)
cat("is.na(na_example):\n") print(is.na(na_example))
NULL example - represents the absence of a value/object
null_example <- NULL cat("\nNULL example:\n") print(null_example)
cat("is.null(null_example):\n") print(is.null(null_example))
NaN example - represents an undefined mathematical result
nan_example <- 0 / 0 cat("\nNaN example (0/0):\n") print(nan_example)
cat("is.nan(nan_example):\n") print(is.nan(nan_example))

```
37
38  ########################################################################
39  # TASK 2: DEMONSTRATE NA, NULL, AND NaN
40  ########################################################################
41
42  cat("\n----- Task 2: NA, NULL, and NaN Demonstration -----\n")
43
44  # NA example - represents a missing value
45  na_example <- NA
46  cat("\nNA example:\n")
47  print(na_example)
48  cat("is.na(na_example):\n")
49  print(is.na(na_example))
50
51  # NULL example - represents the absence of a value/object
52  null_example <- NULL
53  cat("\nNULL example:\n")
54  print(null_example)
55  cat("is.null(null_example):\n")
56  print(is.null(null_example))
57
58  # NaN example - represents an undefined mathematical result
59  nan_example <- 0 / 0
60  cat("\nNaN example (0/0):\n")
61  print(nan_example)
62  cat("is.nan(nan_example):\n")
63  print(is.nan(nan_example))
64
```



Task 3

Created the missing_summary() function to calculate:

- Total Records
- Missing Values
- Missing Percentage

Generated warnings for variables with more than 20% missing values.

```
################################################################################
############
# TASK 3: MISSING VALUE SUMMARY FUNCTION
################################################################################
############

cat("\n----- Task 3: Missing Value Summary -----\n")

missing_summary <- function(data) {

  variables_to_check <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
```

```
  variable_names       <- character(0)
  total_records_vector <- numeric(0)
  missing_values_vector<- numeric(0)
  missing_percent_vector <- numeric(0)

  for (variable in variables_to_check) {
    if (variable %in% names(data)) {
      total_records   <- length(data[[variable]])
      missing_values  <- sum(is.na(data[[variable]]))
      missing_percent <- (missing_values / total_records) * 100

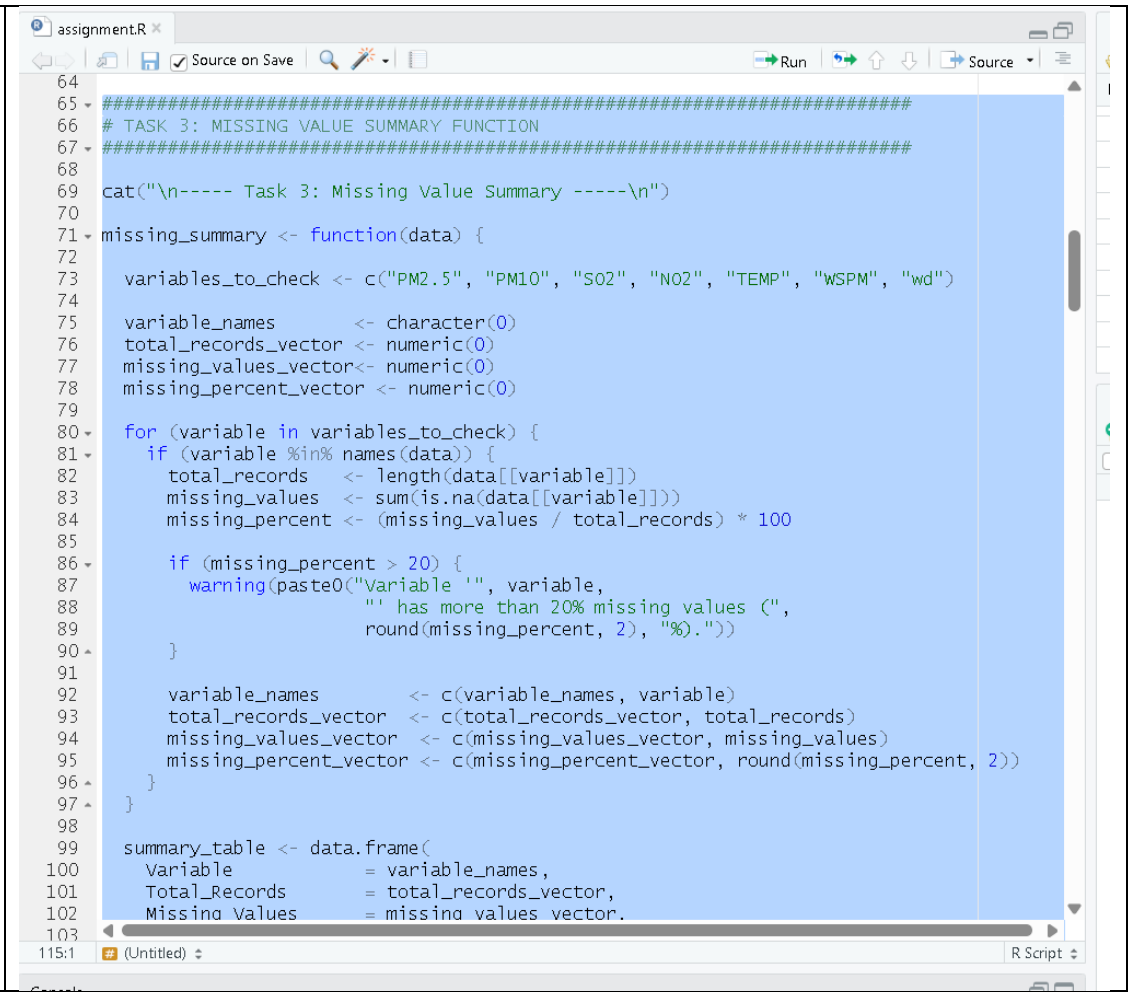
      if (missing_percent > 20) {
        warning(paste0("Variable '", variable,
                       "' has more than 20% missing values (",
                       round(missing_percent, 2), "%)."))
      }

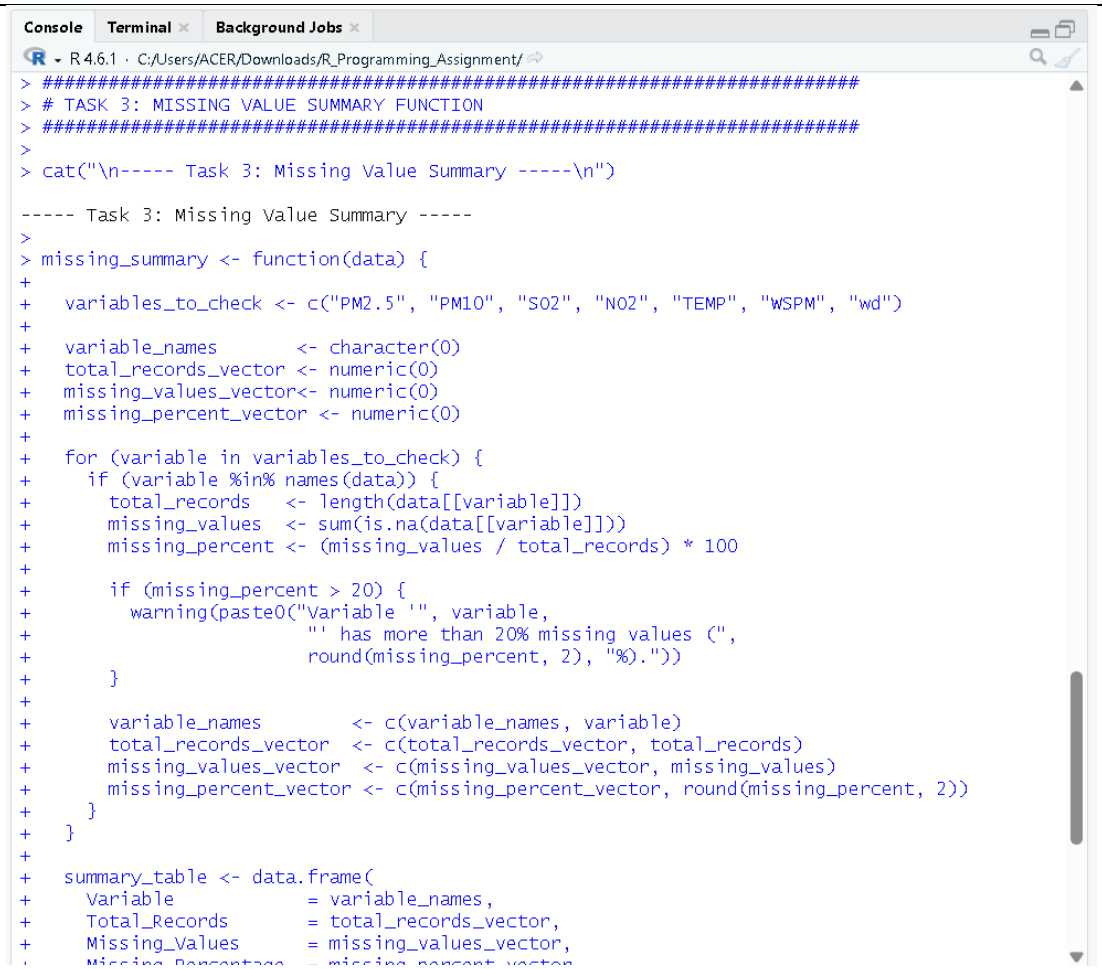
      variable_names        <- c(variable_names, variable)
      total_records_vector  <- c(total_records_vector, total_records)
      missing_values_vector  <- c(missing_values_vector, missing_values)
      missing_percent_vector <- c(missing_percent_vector, round(missing_percent, 2))
    }
  }

  summary_table <- data.frame(
    Variable           = variable_names,
    Total_Records      = total_records_vector,
    Missing_Values     = missing_values_vector,
    Missing_Percentage  = missing_percent_vector,
    stringsAsFactors    = FALSE
  )

  return(summary_table)
}

if (!is.null(air_quality_data)) {
  missing_summary_table <- missing_summary(air_quality_data)
  cat("\nMissing Value Summary Table:\n")
  print(missing_summary_table)
}
```


assignment.R
Source on Save
Run
Source
64
65 ##
66 # TASK 3: MISSING VALUE SUMMARY FUNCTION
67 ##
68
69 cat("\n----- Task 3: Missing Value Summary -----\n")
70
71 missing_summary <- function(data) {
72
73 variables_to_check <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
74
75 variable_names <- character(0)
76 total_records_vector <- numeric(0)
77 missing_values_vector<- numeric(0)
78 missing_percent_vector <- numeric(0)
79
80 for (variable in variables_to_check) {
81 if (variable %in% names(data)) {
82 total_records <- length(data[[variable]])
83 missing_values <- sum(is.na(data[[variable]]))
84 missing_percent <- (missing_values / total_records) * 100
85
86 if (missing_percent > 20) {
87 warning(paste0("Variable '", variable,
88 "' has more than 20% missing values (",
89 round(missing_percent, 2), "%)."))
90 }
91
92 variable_names <- c(variable_names, variable)
93 total_records_vector <- c(total_records_vector, total_records)
94 missing_values_vector <- c(missing_values_vector, missing_values)
95 missing_percent_vector <- c(missing_percent_vector, round(missing_percent, 2))
96 }
97 }
98
99 summary_table <- data.frame(
100 Variable = variable_names,
101 Total_Records = total_records_vector,
102 Missing_Values = missing_values_vector,
103
115:1
(Untitled)
R Script



```
Console   Terminal   Background Jobs
R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/
> ################################################################################
> # TASK 3: MISSING VALUE SUMMARY FUNCTION
> ################################################################################
>
> cat("\n----- Task 3: Missing Value Summary -----\n")

----- Task 3: Missing Value Summary -----
>
> missing_summary <- function(data) {
+
+   variables_to_check <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
+
+   variable_names       <- character(0)
+   total_records_vector <- numeric(0)
+   missing_values_vector<- numeric(0)
+   missing_percent_vector <- numeric(0)
+
+   for (variable in variables_to_check) {
+     if (variable %in% names(data)) {
+       total_records   <- length(data[[variable]])
+       missing_values  <- sum(is.na(data[[variable]]))
+       missing_percent <- (missing_values / total_records) * 100
+
+       if (missing_percent > 20) {
+         warning(paste0("Variable '", variable,
+                        "' has more than 20% missing values (",
+                        round(missing_percent, 2), "%)."))
+       }
+
+       variable_names        <- c(variable_names, variable)
+       total_records_vector  <- c(total_records_vector, total_records)
+       missing_values_vector  <- c(missing_values_vector, missing_values)
+       missing_percent_vector <- c(missing_percent_vector, round(missing_percent, 2))
+     }
+   }
+
+   summary_table <- data.frame(
+     Variable           = variable_names,
+     Total_Records      = total_records_vector,
+     Missing_Values     = missing_values_vector,
```

```
> if (!is.null(air_quality_data)) {
+   missing_summary_table <- missing_summary(air_quality_data)
+   cat("\nMissing Value Summary Table:\n")
+   print(missing_summary_table)
+ }

Missing Value Summary Table:
  Variable Total_Records Missing_Values Missing_Percentage
1    PM2.5         35064            925                2.64
2     PM10         35064            718                2.05
3      SO2         35064            935                2.67
4      NO2         35064           1023                2.92
5     TEMP         35064             20                0.06
6     WSPM         35064             14                0.04
7       wd         35064             81                0.23
```

Task 4

Created the pollution_ratio variable and identified NA, NaN, and Infinite values. Invalid numerical values were replaced with NA.

##
##########

```
# TASK 4: POLLUTION RATIO - HANDLING NA, NaN, AND INFINITE VALUES
##################################################################
###########

cat("\n----- Task 4: Pollution Ratio Creation -----\n")

if (!is.null(air_quality_data)) {
  air_quality_data$pollution_ratio <- air_quality_data$PM2.5 / air_quality_data$PM10

  cat("\nCount of NA values in pollution_ratio (before cleaning):\n")
  print(sum(is.na(air_quality_data$pollution_ratio)))

  cat("\nCount of NaN values in pollution_ratio (before cleaning):\n")
  print(sum(is.nan(air_quality_data$pollution_ratio)))

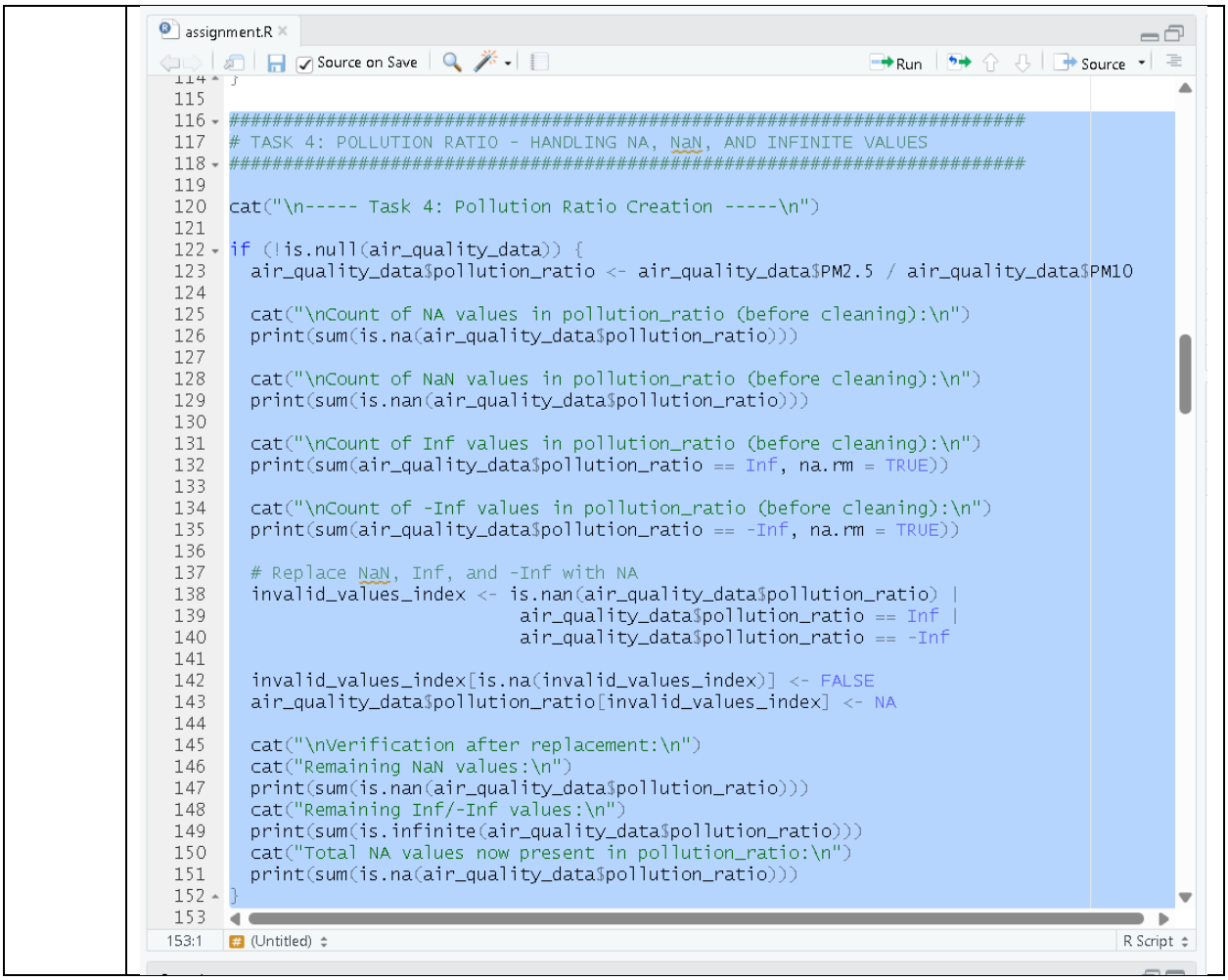
  cat("\nCount of Inf values in pollution_ratio (before cleaning):\n")
  print(sum(air_quality_data$pollution_ratio == Inf, na.rm = TRUE))

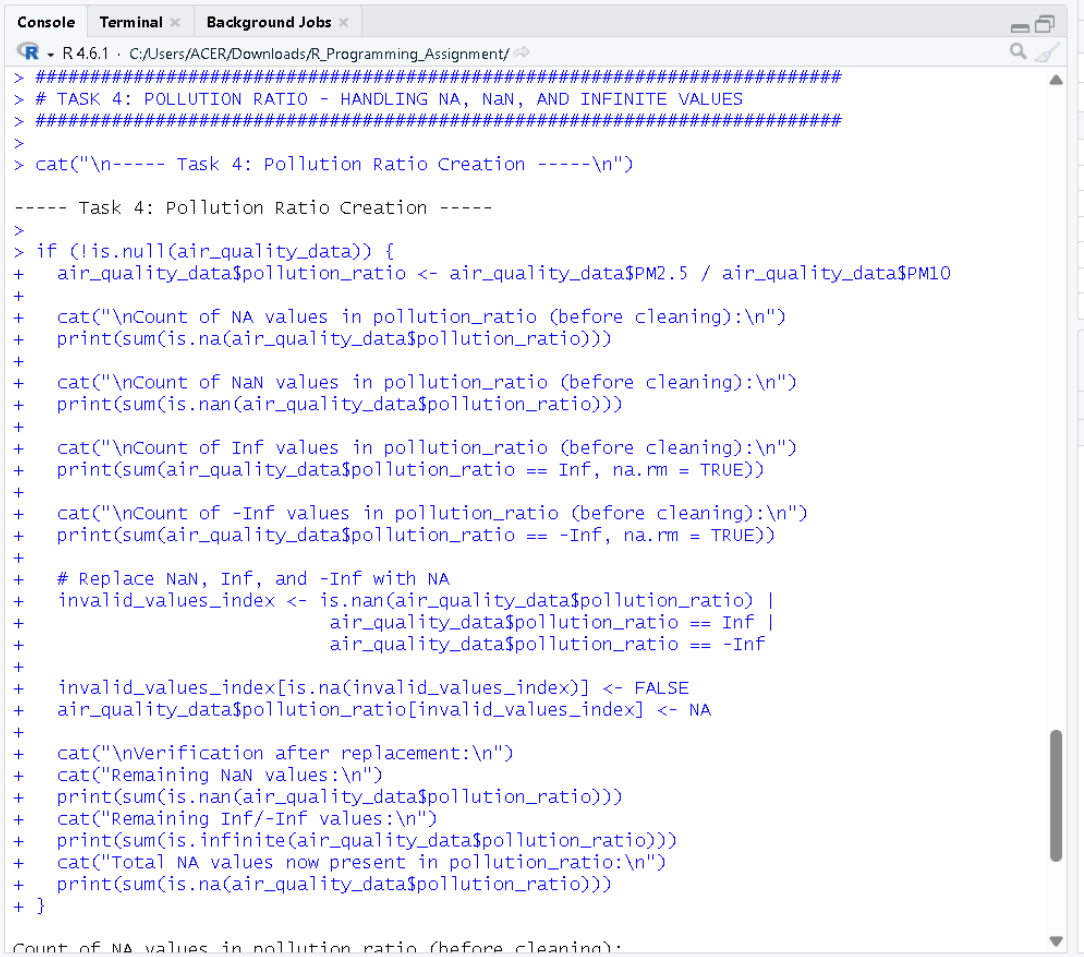
  cat("\nCount of -Inf values in pollution_ratio (before cleaning):\n")
  print(sum(air_quality_data$pollution_ratio == -Inf, na.rm = TRUE))

  # Replace NaN, Inf, and -Inf with NA
  invalid_values_index <- is.nan(air_quality_data$pollution_ratio) |
                          air_quality_data$pollution_ratio == Inf |
                          air_quality_data$pollution_ratio == -Inf

  invalid_values_index[is.na(invalid_values_index)] <- FALSE
  air_quality_data$pollution_ratio[invalid_values_index] <- NA

  cat("\nVerification after replacement:\n")
  cat("Remaining NaN values:\n")
  print(sum(is.nan(air_quality_data$pollution_ratio)))
  cat("Remaining Inf/-Inf values:\n")
  print(sum(is.infinite(air_quality_data$pollution_ratio)))
  cat("Total NA values now present in pollution_ratio:\n")
  print(sum(is.na(air_quality_data$pollution_ratio)))
}
```


assignment.R
Source on Save
Run
Source
}
###
TASK 4: POLLUTION RATIO - HANDLING NA, NaN, AND INFINITE VALUES
###
cat("\n----- Task 4: Pollution Ratio Creation -----\n")
if (!is.null(air_quality_data)) {
air_quality_data$pollution_ratio <- air_quality_data$PM2.5 / air_quality_data$PM10
cat("\nCount of NA values in pollution_ratio (before cleaning):\n")
print(sum(is.na(air_quality_data$pollution_ratio)))
cat("\nCount of NaN values in pollution_ratio (before cleaning):\n")
print(sum(is.nan(air_quality_data$pollution_ratio)))
cat("\nCount of Inf values in pollution_ratio (before cleaning):\n")
print(sum(air_quality_data$pollution_ratio == Inf, na.rm = TRUE))
cat("\nCount of -Inf values in pollution_ratio (before cleaning):\n")
print(sum(air_quality_data$pollution_ratio == -Inf, na.rm = TRUE))
Replace NaN, Inf, and -Inf with NA
invalid_values_index <- is.nan(air_quality_data$pollution_ratio) |
air_quality_data$pollution_ratio == Inf |
air_quality_data$pollution_ratio == -Inf
invalid_values_index[is.na(invalid_values_index)] <- FALSE
air_quality_data$pollution_ratio[invalid_values_index] <- NA
cat("\nVerification after replacement:\n")
cat("Remaining NaN values:\n")
print(sum(is.nan(air_quality_data$pollution_ratio)))
cat("Remaining Inf/-Inf values:\n")
print(sum(is.infinite(air_quality_data$pollution_ratio)))
cat("Total NA values now present in pollution_ratio:\n")
print(sum(is.na(air_quality_data$pollution_ratio)))
}
153:1
(Untitled)
R Script



Task 5

Used a for loop to replace missing numerical values using the median.

```
########################################################################
# TASK 5: REPLACE MISSING NUMERICAL VALUES USING MEDIAN (SINGLE FOR LOOP)
########################################################################

cat("\n----- Task 5: Median Replacement for Numerical Variables -----\n")
```

```
numeric_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM")

if (!is.null(air_quality_data)) {
  for (variable in numeric_variables) {

    if (variable %in% names(air_quality_data)) {

      missing_before <- sum(is.na(air_quality_data[[variable]]))

      median_value <- median(air_quality_data[[variable]], na.rm = TRUE)

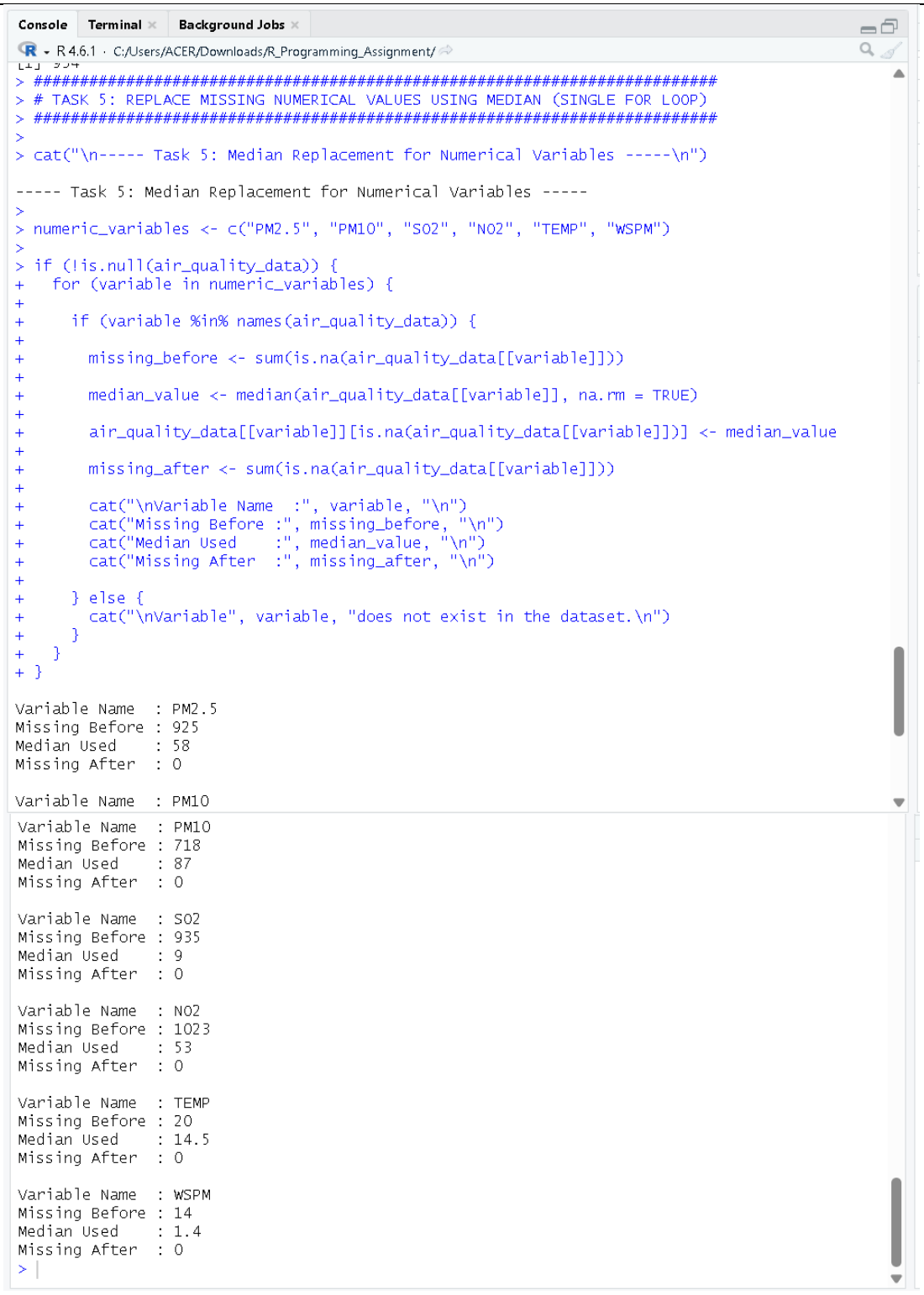
      air_quality_data[[variable]][is.na(air_quality_data[[variable]])] <- median_value

      missing_after <- sum(is.na(air_quality_data[[variable]]))

      cat("\nVariable Name  :", variable, "\n")
      cat("Missing Before :", missing_before, "\n")
      cat("Median Used    :", median_value, "\n")
      cat("Missing After  :", missing_after, "\n")

    } else {
      cat("\nVariable", variable, "does not exist in the dataset.\n")
    }
  }
}
```

```
154 ##########################################################################
155 # TASK 5: REPLACE MISSING NUMERICAL VALUES USING MEDIAN (SINGLE FOR LOOP)
156 ##########################################################################
157
158 cat("\n----- Task 5: Median Replacement for Numerical Variables -----\n")
159
160 numeric_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM")
161
162 if (!is.null(air_quality_data)) {
163   for (variable in numeric_variables) {
164
165     if (variable %in% names(air_quality_data)) {
166
167       missing_before <- sum(is.na(air_quality_data[[variable]]))
168
169       median_value <- median(air_quality_data[[variable]], na.rm = TRUE)
170
171       air_quality_data[[variable]][is.na(air_quality_data[[variable]])] <- median_value
172
173       missing_after <- sum(is.na(air_quality_data[[variable]]))
174
175       cat("\nVariable Name  :", variable, "\n")
176       cat("Missing Before :", missing_before, "\n")
177       cat("Median Used    :", median_value, "\n")
178       cat("Missing After  :", missing_after, "\n")
179
180     } else {
181       cat("\nVariable", variable, "does not exist in the dataset.\n")
182     }
183   }
184 }
185
```



Task 6

Created the calculate_mode() function and replaced missing values in the categorical variable wd.

```
#################################################################
###########
# TASK 6: MODE CALCULATION AND REPLACEMENT FOR CATEGORICAL VARIABLE
(wd)
#################################################################
###########

cat("\n----- Task 6: Mode Replacement for Categorical Variable (wd) -----\n")

calculate_mode <- function(x) {
  x <- x[!is.na(x)]
  unique_values <- unique(x)
  frequency_counts <- tabulate(match(x, unique_values))
  mode_value <- unique_values[which.max(frequency_counts)]
  return(mode_value)
}

if (!is.null(air_quality_data) && "wd" %in% names(air_quality_data)) {

  missing_before_wd <- sum(is.na(air_quality_data$wd))

  mode_wd <- calculate_mode(air_quality_data$wd)

  air_quality_data$wd[is.na(air_quality_data$wd)] <- mode_wd

  missing_after_wd <- sum(is.na(air_quality_data$wd))

  cat("\nMissing Before :", missing_before_wd, "\n")
  cat("Mode Used      :", mode_wd, "\n")
  cat("Missing After  :", missing_after_wd, "\n")
}
```

```r
################################################################################
# TASK 6: MODE CALCULATION AND REPLACEMENT FOR CATEGORICAL VARIABLE (wd)
################################################################################

cat("\n----- Task 6: Mode Replacement for Categorical Variable (wd) -----\n")

calculate_mode <- function(x) {
  x <- x[!is.na(x)]
  unique_values <- unique(x)
  frequency_counts <- tabulate(match(x, unique_values))
  mode_value <- unique_values[which.max(frequency_counts)]
  return(mode_value)
}

if (!is.null(air_quality_data) && "wd" %in% names(air_quality_data)) {

  missing_before_wd <- sum(is.na(air_quality_data$wd))

  mode_wd <- calculate_mode(air_quality_data$wd)

  air_quality_data$wd[is.na(air_quality_data$wd)] <- mode_wd

  missing_after_wd <- sum(is.na(air_quality_data$wd))

  cat("\nMissing Before :", missing_before_wd, "\n")
  cat("Mode Used      :", mode_wd, "\n")
  cat("Missing After  :", missing_after_wd, "\n")
}
```

```
> ################################################################################
> # TASK 6: MODE CALCULATION AND REPLACEMENT FOR CATEGORICAL VARIABLE (wd)
> ################################################################################
>
> cat("\n----- Task 6: Mode Replacement for Categorical Variable (wd) -----\n")

----- Task 6: Mode Replacement for Categorical Variable (wd) -----
>
> calculate_mode <- function(x) {
+   x <- x[!is.na(x)]
+   unique_values <- unique(x)
+   frequency_counts <- tabulate(match(x, unique_values))
+   mode_value <- unique_values[which.max(frequency_counts)]
+   return(mode_value)
+ }
>
> if (!is.null(air_quality_data) && "wd" %in% names(air_quality_data)) {
+
+   missing_before_wd <- sum(is.na(air_quality_data$wd))
+
+   mode_wd <- calculate_mode(air_quality_data$wd)
+
+   air_quality_data$wd[is.na(air_quality_data$wd)] <- mode_wd
+
+   missing_after_wd <- sum(is.na(air_quality_data$wd))
+
+   cat("\nMissing Before :", missing_before_wd, "\n")
+   cat("Mode Used      :", mode_wd, "\n")
+   cat("Missing After  :", missing_after_wd, "\n")
+ }

Missing Before : 81
Mode Used      : NE
Missing After  : 0
```

Task 7

Implemented clean_variable() using tryCatch() to handle different error conditions gracefully.

```
################################################################################
# TASK 7: ERROR HANDLING USING tryCatch() - clean_variable()
```

```
##################################################################
###########

cat("\n----- Task 7: Error Handling with clean_variable() -----\n")

clean_variable <- function(data, variable_name) {

  result <- tryCatch({

    if (!(variable_name %in% names(data))) {
      stop(paste0("Variable '", variable_name, "' does not exist in the dataset."))
    }

    column_data <- data[[variable_name]]

    if (!is.numeric(column_data)) {
      stop(paste0("Variable '", variable_name, "' is not numeric."))
    }

    if (all(is.na(column_data))) {
      stop(paste0("Variable '", variable_name, "' contains only missing values."))
    }

    median_value <- median(column_data, na.rm = TRUE)

    if (is.na(median_value)) {
      stop(paste0("Median could not be calculated for variable '", variable_name, "'."))
    }

    column_data[is.na(column_data)] <- median_value
    data[[variable_name]] <- column_data

    cat("Variable '", variable_name, "' cleaned successfully using median = ",
        median_value, "\n", sep = "")

    return(data)

  }, error = function(e) {
    cat("Error while cleaning variable '", variable_name, "': ",
        conditionMessage(e), "\n", sep = "")
    return(data)
  })

  return(result)
}
```

```
if (!is.null(air_quality_data)) {

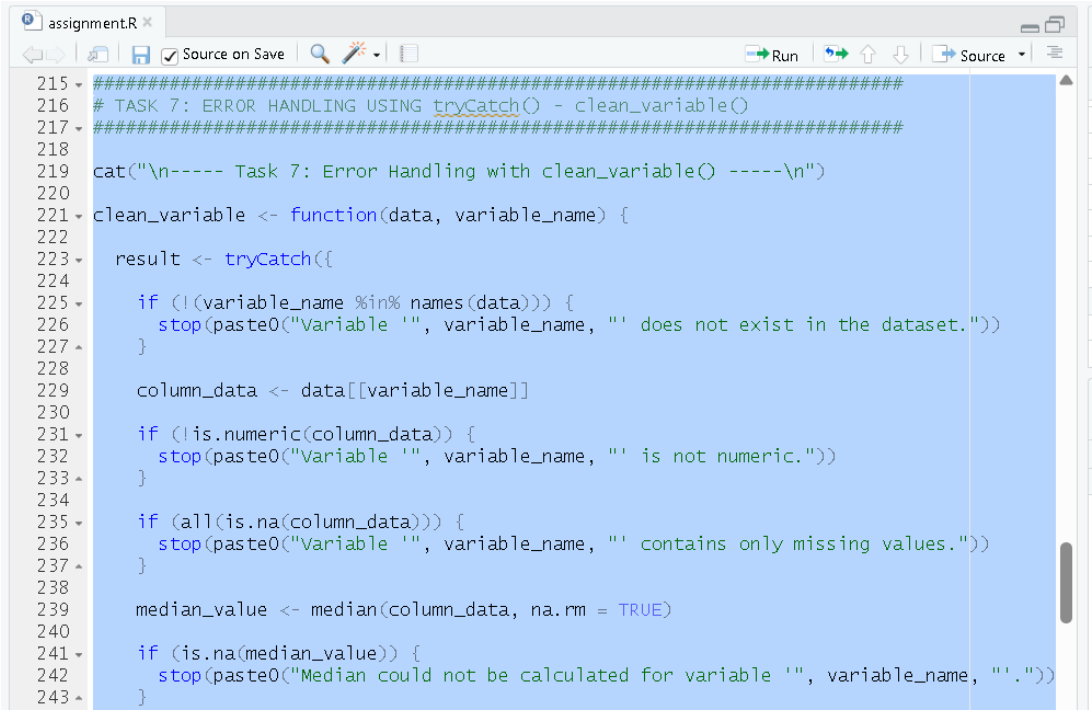
  cat("\nCase 1: Variable does not exist\n")
  air_quality_data <- clean_variable(air_quality_data, "OZONE_LEVEL")

  cat("\nCase 2: Variable is not numeric\n")
  air_quality_data <- clean_variable(air_quality_data, "wd")

  cat("\nCase 3: Variable contains only missing values\n")
  air_quality_data$all_missing_test <- rep(NA_real_, nrow(air_quality_data))
  air_quality_data <- clean_variable(air_quality_data, "all_missing_test")

  cat("\nCase 4: Median cannot be calculated (empty numeric vector simulation)\n")
  air_quality_data$empty_numeric_test <- rep(NaN, nrow(air_quality_data))
  air_quality_data <- clean_variable(air_quality_data, "empty_numeric_test")

  air_quality_data$all_missing_test  <- NULL
  air_quality_data$empty_numeric_test <- NULL
}
```



```r
    }

    column_data[is.na(column_data)] <- median_value
    data[[variable_name]] <- column_data

    cat("Variable '", variable_name, "' cleaned successfully using median = ",
        median_value, "\n", sep = "")

    return(data)

  }, error = function(e) {
    cat("Error while cleaning variable '", variable_name, "': ",
        conditionMessage(e), "\n", sep = "")
    return(data)
  })

  return(result)
}

if (!is.null(air_quality_data)) {

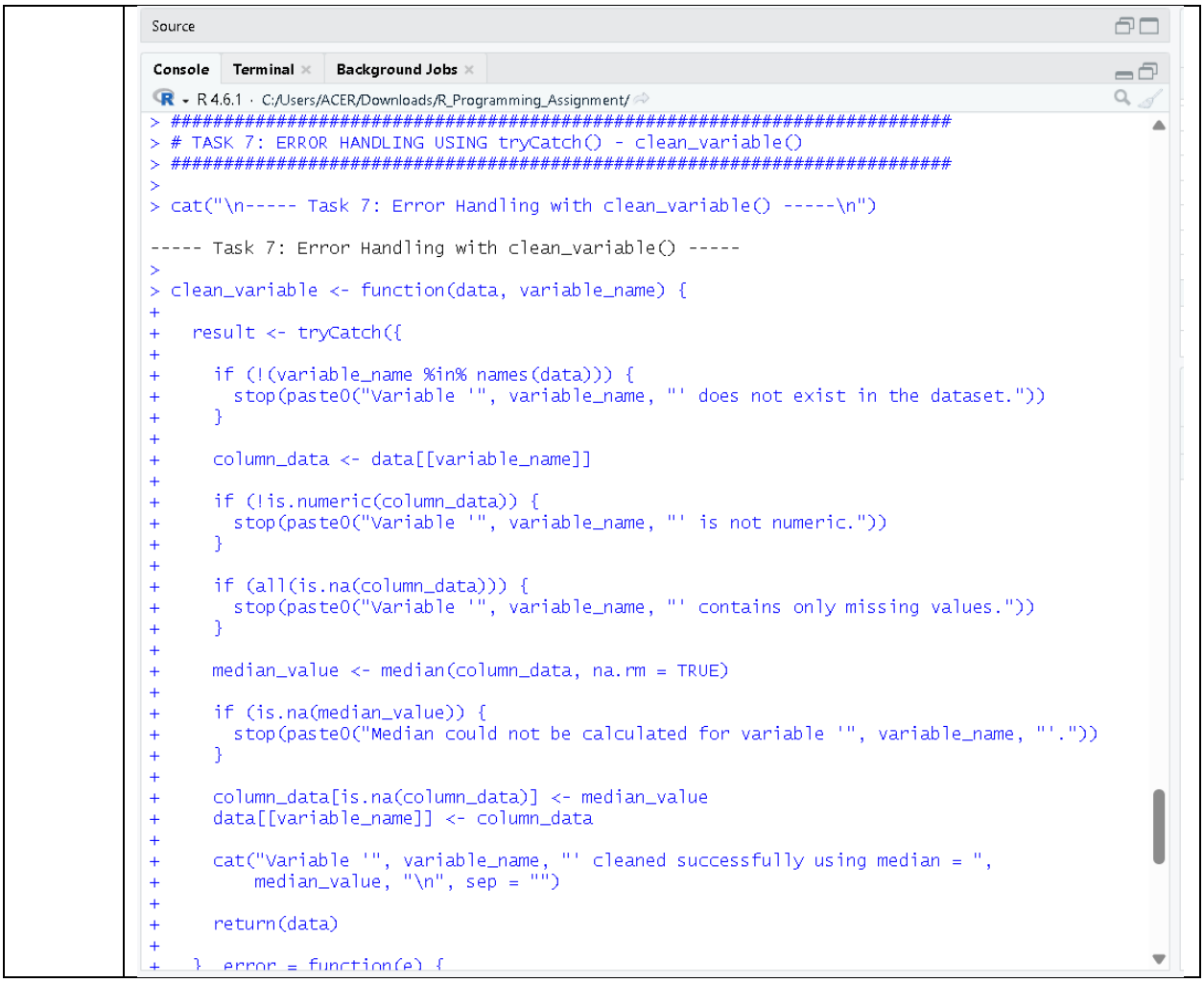
  cat("\nCase 1: Variable does not exist\n")
  air_quality_data <- clean_variable(air_quality_data, "OZONE_LEVEL")

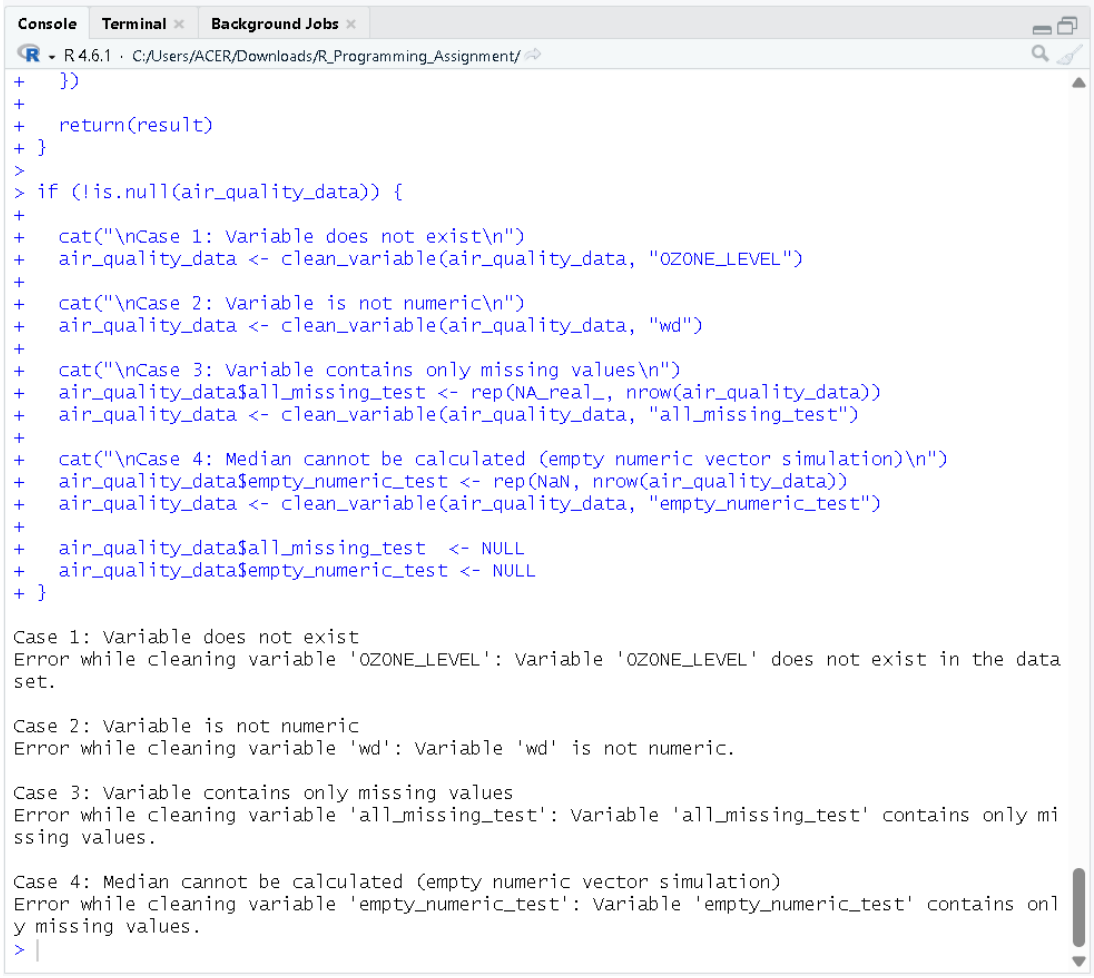
  cat("\nCase 2: Variable is not numeric\n")
  air_quality_data <- clean_variable(air_quality_data, "wd")

  cat("\nCase 3: Variable contains only missing values\n")
  air_quality_data$all_missing_test <- rep(NA_real_, nrow(air_quality_data))
  air_quality_data <- clean_variable(air_quality_data, "all_missing_test")

  cat("\nCase 4: Median cannot be calculated (empty numeric vector simulation)\n")
  air_quality_data$empty_numeric_test <- rep(NaN, nrow(air_quality_data))
  air_quality_data <- clean_variable(air_quality_data, "empty_numeric_test")

  air_quality_data$all_missing_test  <- NULL
  air_quality_data$empty_numeric_test <- NULL
}
```


Source
Console
Terminal
Background Jobs
R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/
> ##
> # TASK 7: ERROR HANDLING USING tryCatch() - clean_variable()
> ##
>
> cat("\n----- Task 7: Error Handling with clean_variable() -----\n")
----- Task 7: Error Handling with clean_variable() -----
>
> clean_variable <- function(data, variable_name) {
+
+   result <- tryCatch({
+
+   if (!(variable_name %in% names(data))) {
+   stop(paste0("Variable '", variable_name, "' does not exist in the dataset."))
+   }
+
+   column_data <- data[[variable_name]]
+
+   if (!is.numeric(column_data)) {
+   stop(paste0("Variable '", variable_name, "' is not numeric."))
+   }
+
+   if (all(is.na(column_data))) {
+   stop(paste0("Variable '", variable_name, "' contains only missing values."))
+   }
+
+   median_value <- median(column_data, na.rm = TRUE)
+
+   if (is.na(median_value)) {
+   stop(paste0("Median could not be calculated for variable '", variable_name, "'."))
+   }
+
+   column_data[is.na(column_data)] <- median_value
+   data[[variable_name]] <- column_data
+
+   cat("Variable '", variable_name, "' cleaned successfully using median = ",
+   median_value, "\n", sep = "")
+
+   return(data)
+
+   }, error = function(e) {



Task 8

Prepared a comparison table showing missing values before and after cleaning.

```
##################################################################
############
# TASK 8: COMPARISON TABLE - MISSING VALUES BEFORE VS AFTER CLEANING
##################################################################
############

cat("\n----- Task 8: Comparison Table (Before vs After Cleaning) -----\n")

original_data_for_comparison <- tryCatch({
  read.csv(file_path, stringsAsFactors = FALSE)
}, error = function(e) {
  message("Error while reloading the original file: ", conditionMessage(e))
  return(NULL)
})

comparison_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
```

```
if (!is.null(original_data_for_comparison) && !is.null(air_quality_data)) {

  variable_names_comp  <- character(0)
  missing_before_vector <- numeric(0)
  missing_after_vector  <- numeric(0)

  for (variable in comparison_variables) {
    if (variable %in% names(original_data_for_comparison) &&
        variable %in% names(air_quality_data)) {

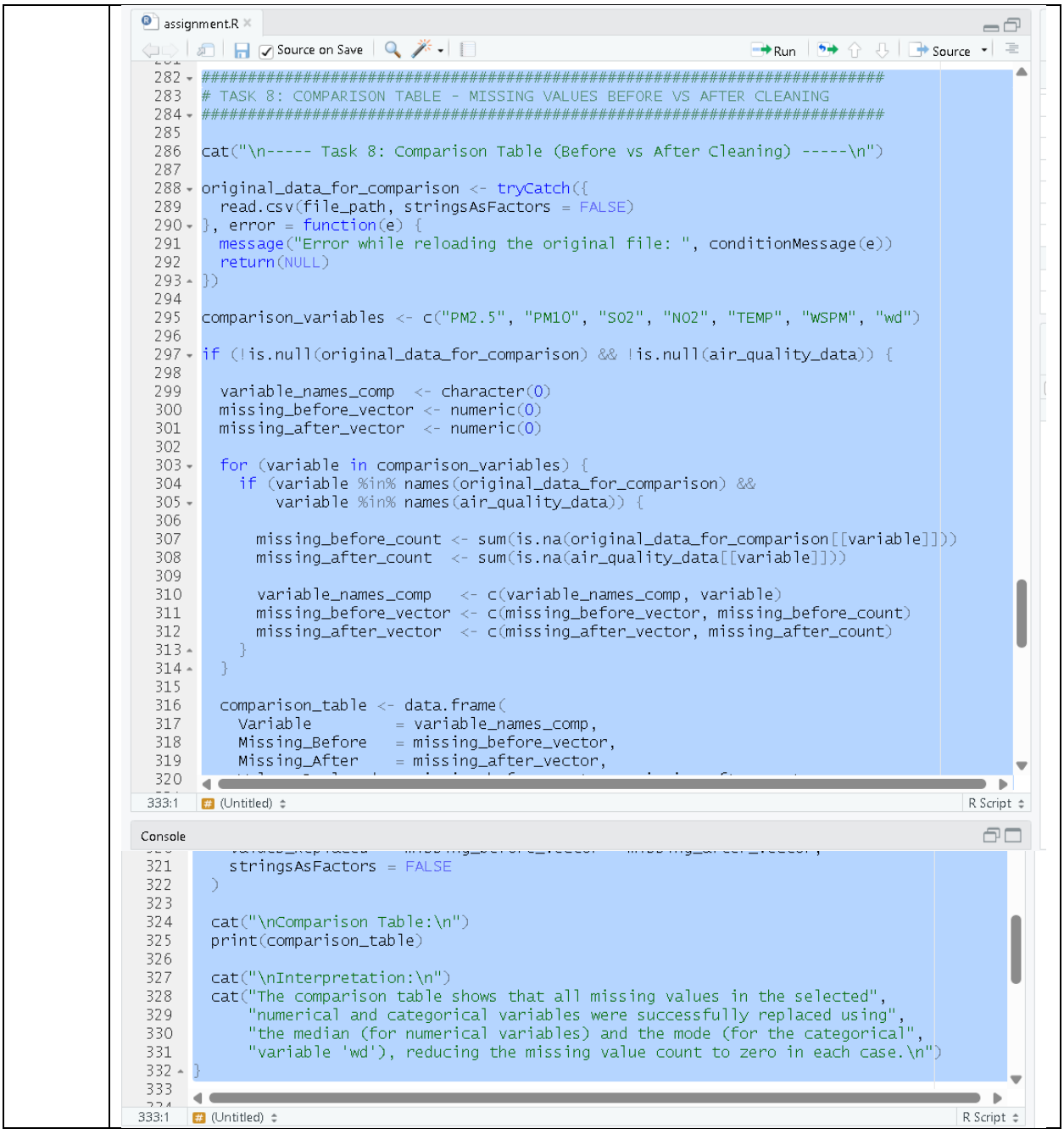
      missing_before_count <- sum(is.na(original_data_for_comparison[[variable]]))
      missing_after_count  <- sum(is.na(air_quality_data[[variable]]))

      variable_names_comp   <- c(variable_names_comp, variable)
      missing_before_vector <- c(missing_before_vector, missing_before_count)
      missing_after_vector  <- c(missing_after_vector, missing_after_count)
    }
  }

  comparison_table <- data.frame(
    Variable         = variable_names_comp,
    Missing_Before   = missing_before_vector,
    Missing_After    = missing_after_vector,
    Values_Replaced  = missing_before_vector - missing_after_vector,
    stringsAsFactors = FALSE
  )

  cat("\nComparison Table:\n")
  print(comparison_table)

  cat("\nInterpretation:\n")
  cat("The comparison table shows that all missing values in the selected",
      "numerical and categorical variables were successfully replaced using",
      "the median (for numerical variables) and the mode (for the categorical",
      "variable 'wd'), reducing the missing value count to zero in each case.\n")
}
```


assignment.R
Source on Save
Run
Source
282 ##
283 # TASK 8: COMPARISON TABLE - MISSING VALUES BEFORE VS AFTER CLEANING
284 ##
285
286 cat("\n----- Task 8: Comparison Table (Before vs After Cleaning) -----\n")
287
288 original_data_for_comparison <- tryCatch({
289 read.csv(file_path, stringsAsFactors = FALSE)
290 }, error = function(e) {
291 message("Error while reloading the original file: ", conditionMessage(e))
292 return(NULL)
293 })
294
295 comparison_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
296
297 if (!is.null(original_data_for_comparison) && !is.null(air_quality_data)) {
298
299 variable_names_comp <- character(0)
300 missing_before_vector <- numeric(0)
301 missing_after_vector <- numeric(0)
302
303 for (variable in comparison_variables) {
304 if (variable %in% names(original_data_for_comparison) &&
305 variable %in% names(air_quality_data)) {
306
307 missing_before_count <- sum(is.na(original_data_for_comparison[[variable]]))
308 missing_after_count <- sum(is.na(air_quality_data[[variable]]))
309
310 variable_names_comp <- c(variable_names_comp, variable)
311 missing_before_vector <- c(missing_before_vector, missing_before_count)
312 missing_after_vector <- c(missing_after_vector, missing_after_count)
313 }
314 }
315
316 comparison_table <- data.frame(
317 Variable = variable_names_comp,
318 Missing_Before = missing_before_vector,
319 Missing_After = missing_after_vector,
320
333:1
(Untitled)
R Script
Console
321 stringsAsFactors = FALSE
322)
323
324 cat("\nComparison Table:\n")
325 print(comparison_table)
326
327 cat("\nInterpretation:\n")
328 cat("The comparison table shows that all missing values in the selected",
329 "numerical and categorical variables were successfully replaced using",
330 "the median (for numerical variables) and the mode (for the categorical",
331 "variable 'wd'), reducing the missing value count to zero in each case.\n")
332 }
333
333:1
(Untitled)
R Script

Console | Terminal | Background Jobs

R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/

```
> ############################################################################
> # TASK 8: COMPARISON TABLE - MISSING VALUES BEFORE VS AFTER CLEANING
> ############################################################################
>
> cat("\n----- Task 8: Comparison Table (Before vs After Cleaning) -----\n")

----- Task 8: Comparison Table (Before vs After Cleaning) -----
>
> original_data_for_comparison <- tryCatch({
+   read.csv(file_path, stringsAsFactors = FALSE)
+ }, error = function(e) {
+   message("Error while reloading the original file: ", conditionMessage(e))
+   return(NULL)
+ })
>
> comparison_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
>
> if (!is.null(original_data_for_comparison) && !is.null(air_quality_data)) {
+
+   variable_names_comp  <- character(0)
+   missing_before_vector <- numeric(0)
+   missing_after_vector  <- numeric(0)
+
+   for (variable in comparison_variables) {
+     if (variable %in% names(original_data_for_comparison) &&
+         variable %in% names(air_quality_data)) {
+
+       missing_before_count <- sum(is.na(original_data_for_comparison[[variable]]))
+       missing_after_count  <- sum(is.na(air_quality_data[[variable]]))
+
+       variable_names_comp   <- c(variable_names_comp, variable)
+       missing_before_vector <- c(missing_before_vector, missing_before_count)
+       missing_after_vector  <- c(missing_after_vector, missing_after_count)
+     }
+   }
+
+   comparison_table <- data.frame(
+     Variable         = variable_names_comp,
+     Missing_Before   = missing_before_vector,
+     Missing_After    = missing_after_vector,
+     Values_Replaced  = missing_before_vector - missing_after_vector,
+     stringsAsFactors = FALSE
+   )
+
+   cat("\nComparison Table:\n")
+   print(comparison_table)
+
+   cat("\nInterpretation:\n")
+   cat("The comparison table shows that all missing values in the selected",
+       "numerical and categorical variables were successfully replaced using",
+       "the median (for numerical variables) and the mode (for the categorical",
+       "variable 'wd'), reducing the missing value count to zero in each case.\n")
+ }

Comparison Table:
  Variable Missing_Before Missing_After Values_Replaced
1    PM2.5            925             0             925
2     PM10            718             0             718
3      SO2            935             0             935
4      NO2           1023             0            1023
5     TEMP             20             0              20
6     WSPM             14             0              14
7       wd             81             0              81

Interpretation:
The comparison table shows that all missing values in the selected numerical and categorical v
ariables were successfully replaced using the median (for numerical variables) and the mode (f
or the categorical variable 'wd'), reducing the missing value count to zero in each case.
```

Task 9

	Created a grouped bar chart comparing missing values before and after cleaning.
	
	``
	##
	# TASK 9: GROUPED BAR CHART - MISSING VALUES BEFORE VS AFTER CLEANING
	##
	
	cat("\n----- Task 9: Bar Chart Visualization -----\n")
	
	if (exists("comparison_table")) {
	
	  bar_chart_matrix <- rbind(comparison_table$Missing_Before,
	                            comparison_table$Missing_After)
	
	  colnames(bar_chart_matrix) <- comparison_table$Variable
	  rownames(bar_chart_matrix) <- c("Before Cleaning", "After Cleaning")
	
	  png("Missing_Value_BarChart.png", width = 900, height = 600)
	
	  barplot(bar_chart_matrix,
	          beside = TRUE,
	          col = c("firebrick", "forestgreen"),
	          main = "Missing Values Before and After Cleaning",
	          xlab = "Variables",
	          ylab = "Number of Missing Values",
	          legend.text = rownames(bar_chart_matrix),
	          args.legend = list(x = "topright"))
	
	  dev.off()
	
	  cat("\nBar chart saved successfully as Missing_Value_BarChart.png\n")
	}
	``

```r
################################################################################
# TASK 9: GROUPED BAR CHART - MISSING VALUES BEFORE VS AFTER CLEANING
################################################################################

cat("\n----- Task 9: Bar Chart Visualization -----\n")

if (exists("comparison_table")) {

  bar_chart_matrix <- rbind(comparison_table$Missing_Before,
                            comparison_table$Missing_After)

  colnames(bar_chart_matrix) <- comparison_table$Variable
  rownames(bar_chart_matrix) <- c("Before Cleaning", "After Cleaning")

  png("Missing_Value_BarChart.png", width = 900, height = 600)

  barplot(bar_chart_matrix,
          beside = TRUE,
          col = c("firebrick", "forestgreen"),
          main = "Missing Values Before and After Cleaning",
          xlab = "Variables",
          ylab = "Number of Missing Values",
          legend.text = rownames(bar_chart_matrix),
          args.legend = list(x = "topright"))

  dev.off()

  cat("\nBar chart saved successfully as Missing_Value_BarChart.png\n")
}
```

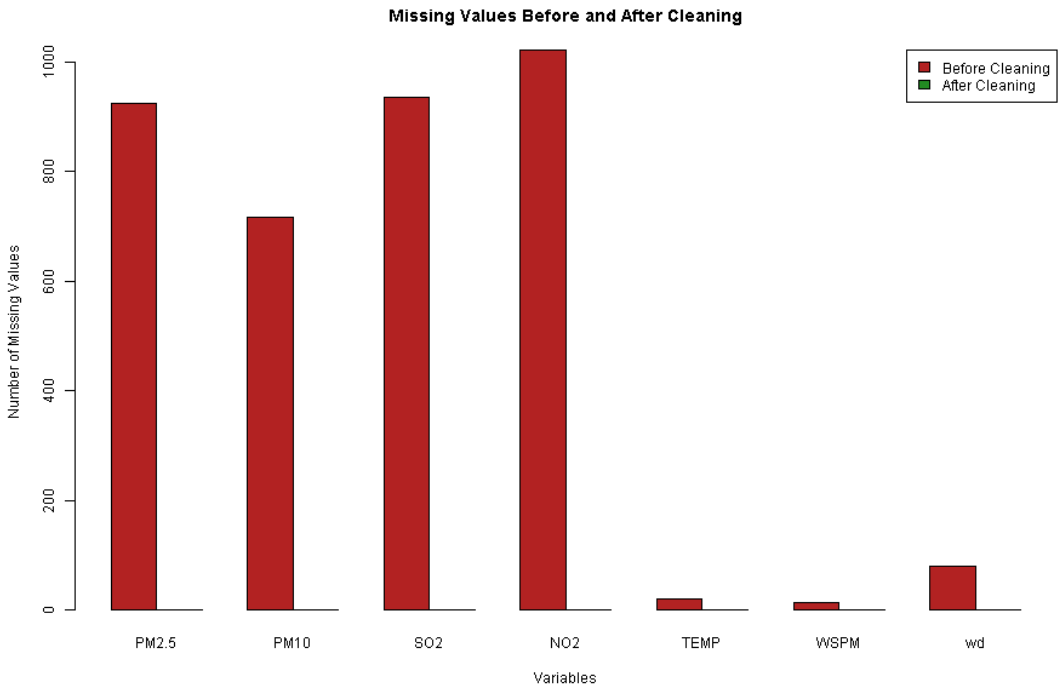
Console | Terminal | Background Jobs

R 4.6.1 · C:/Users/ACER/Downloads/R_Programming_Assignment/

```
> ################################################################################
> # TASK 9: GROUPED BAR CHART - MISSING VALUES BEFORE VS AFTER CLEANING
> ################################################################################
>
> cat("\n----- Task 9: Bar Chart Visualization -----\n")

----- Task 9: Bar Chart Visualization -----
>
> if (exists("comparison_table")) {
+
+   bar_chart_matrix <- rbind(comparison_table$Missing_Before,
+                             comparison_table$Missing_After)
+
+   colnames(bar_chart_matrix) <- comparison_table$Variable
+   rownames(bar_chart_matrix) <- c("Before Cleaning", "After Cleaning")
+
+   png("Missing_Value_BarChart.png", width = 900, height = 600)
+
+   barplot(bar_chart_matrix,
+           beside = TRUE,
+           col = c("firebrick", "forestgreen"),
+           main = "Missing Values Before and After Cleaning",
+           xlab = "Variables",
+           ylab = "Number of Missing Values",
+           legend.text = rownames(bar_chart_matrix),
+           args.legend = list(x = "topright"))
+
+   dev.off()
+
+   cat("\nBar chart saved successfully as Missing_Value_BarChart.png\n")
+ }

Bar chart saved successfully as Missing_Value_BarChart.png
```



Task 10

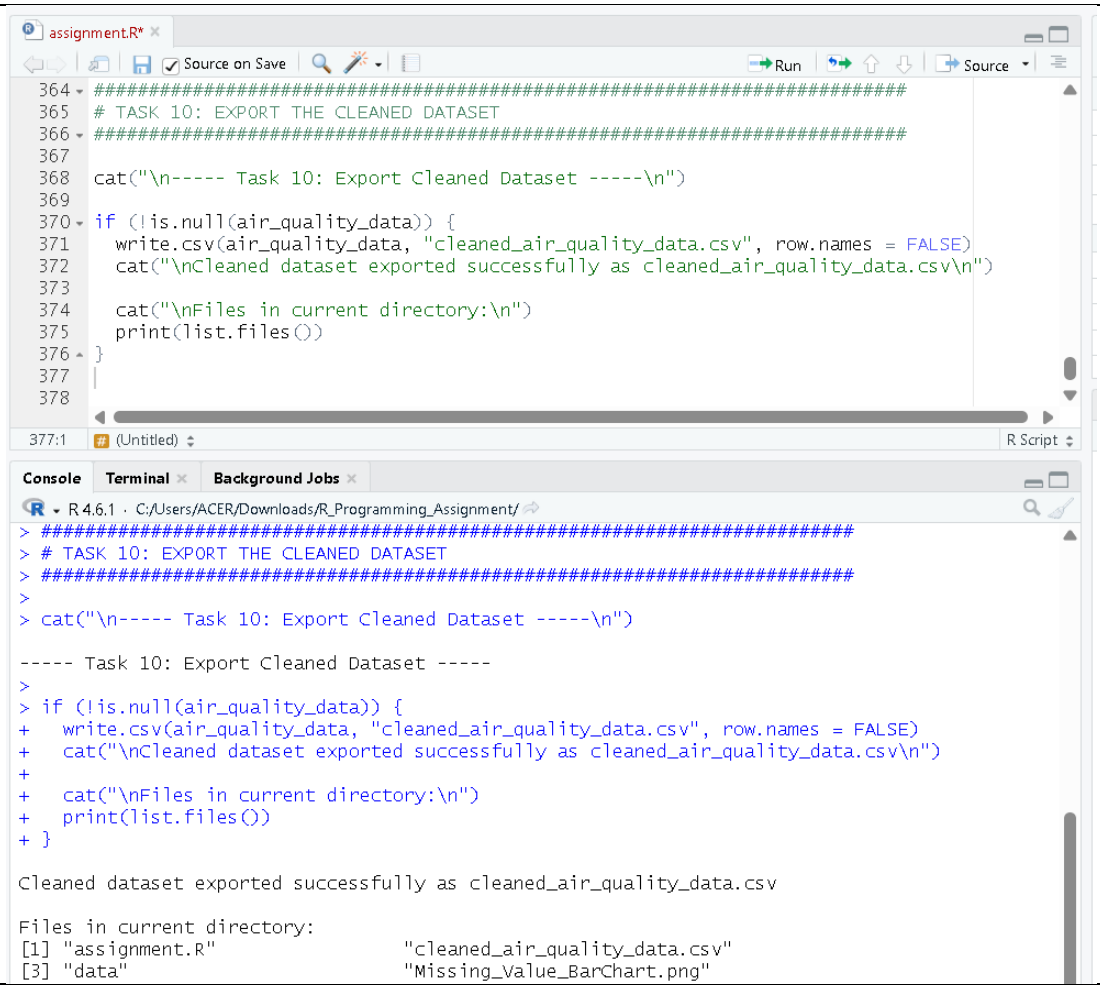
Exported the cleaned dataset as cleaned_air_quality_data.csv.

```
##################################################################
###########
# TASK 10: EXPORT THE CLEANED DATASET
##################################################################
###########

cat("\n----- Task 10: Export Cleaned Dataset -----\n")

if (!is.null(air_quality_data)) {
  write.csv(air_quality_data, "cleaned_air_quality_data.csv", row.names = FALSE)
  cat("\nCleaned dataset exported successfully as cleaned_air_quality_data.csv\n")

  cat("\nFiles in current directory:\n")
  print(list.files())
}
```



Conclusion	This experiment successfully demonstrated how R can be used to clean and prepare a dataset for analysis. Missing values were identified and handled using suitable techniques for both numerical and categorical variables, resulting in a more complete and reliable dataset. The use of user-defined functions, loops, and error handling helped make the program more organised and efficient. The cleaned dataset was then exported, and a bar chart was generated to compare the missing values before and after cleaning. Overall, the experiment highlighted the importance of data preprocessing in improving data quality before performing further analysis or building machine learning models.